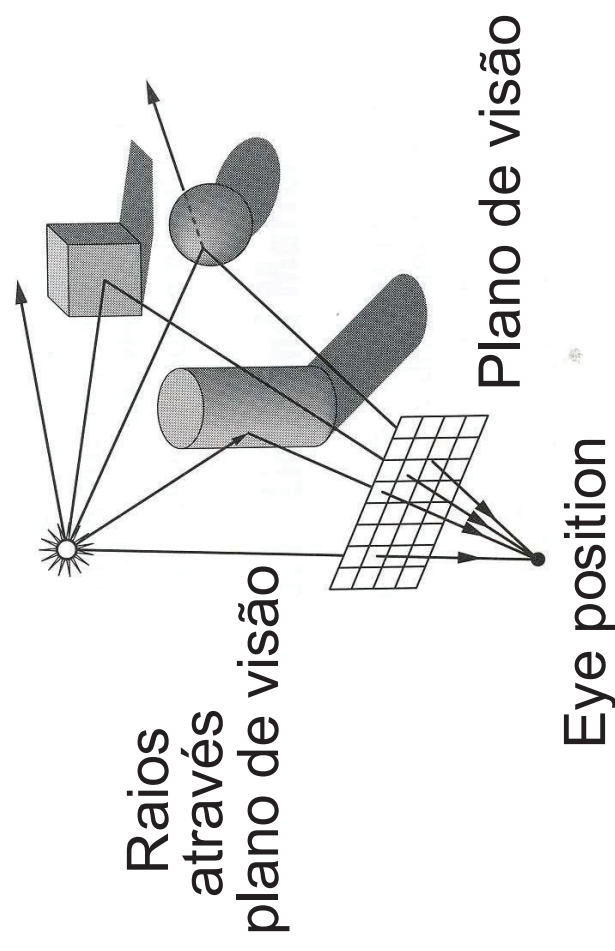


Rendering 3D

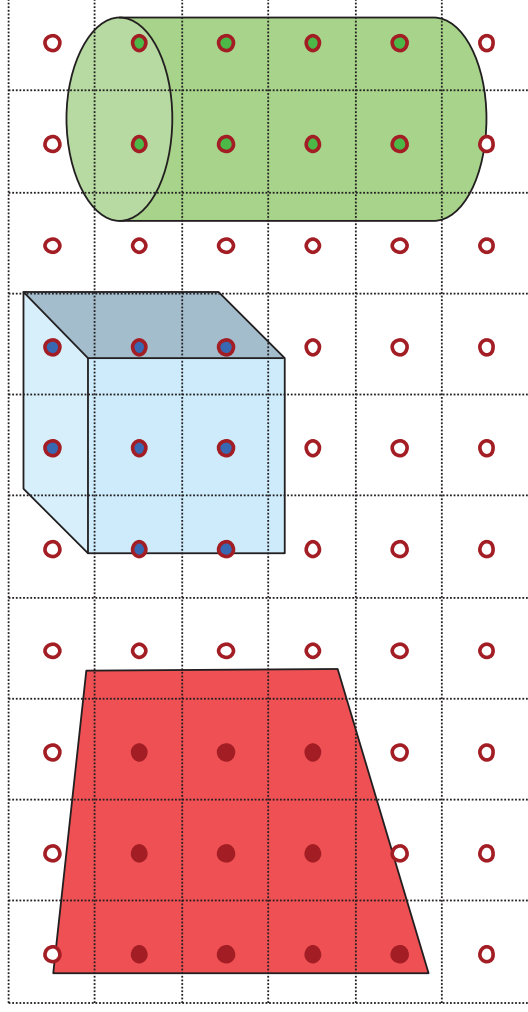
- A cor de cada pixel no plano de visão depende da radiancia que emana das superficies visíveis



Método simples
é ray casting

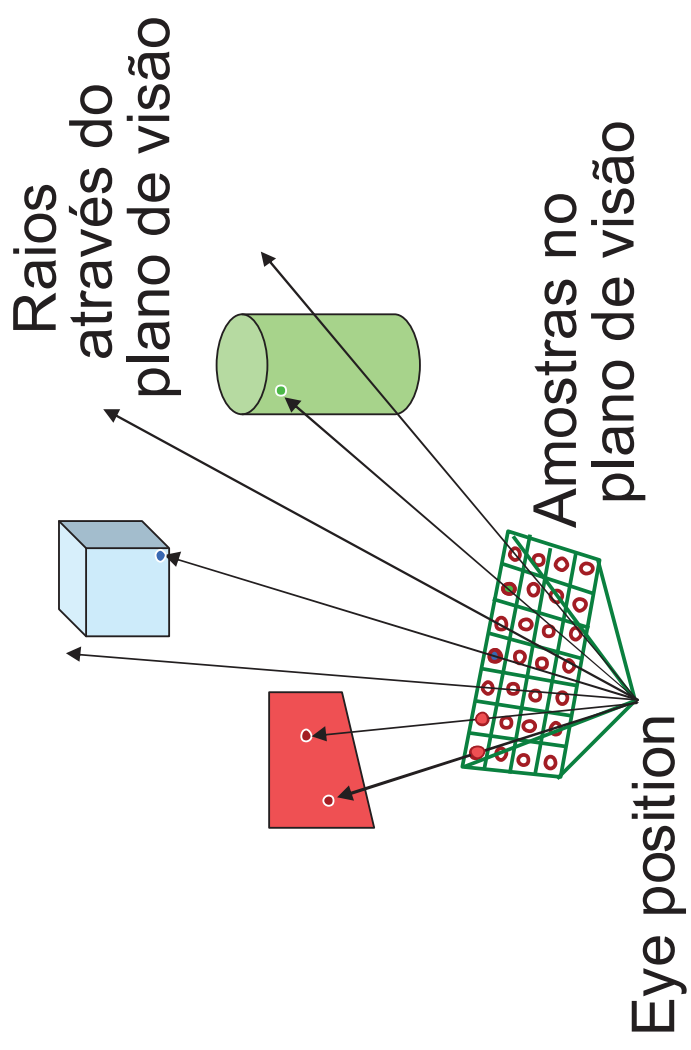
Ray Casting

- Para cada amostra ...
 - Construir raio da câmera através do plano de visão
 - Encontrar a 1a. superfície que intersecta o raio
 - Computar a cor da amostra baseada na radiância



Ray Casting

- Para cada amostra ...
 - Construir raio da câmera através do plano de visão
 - Encontrar a 1a. superfície que intersecta o raio
 - Computar a cor da amostra baseada na radiância

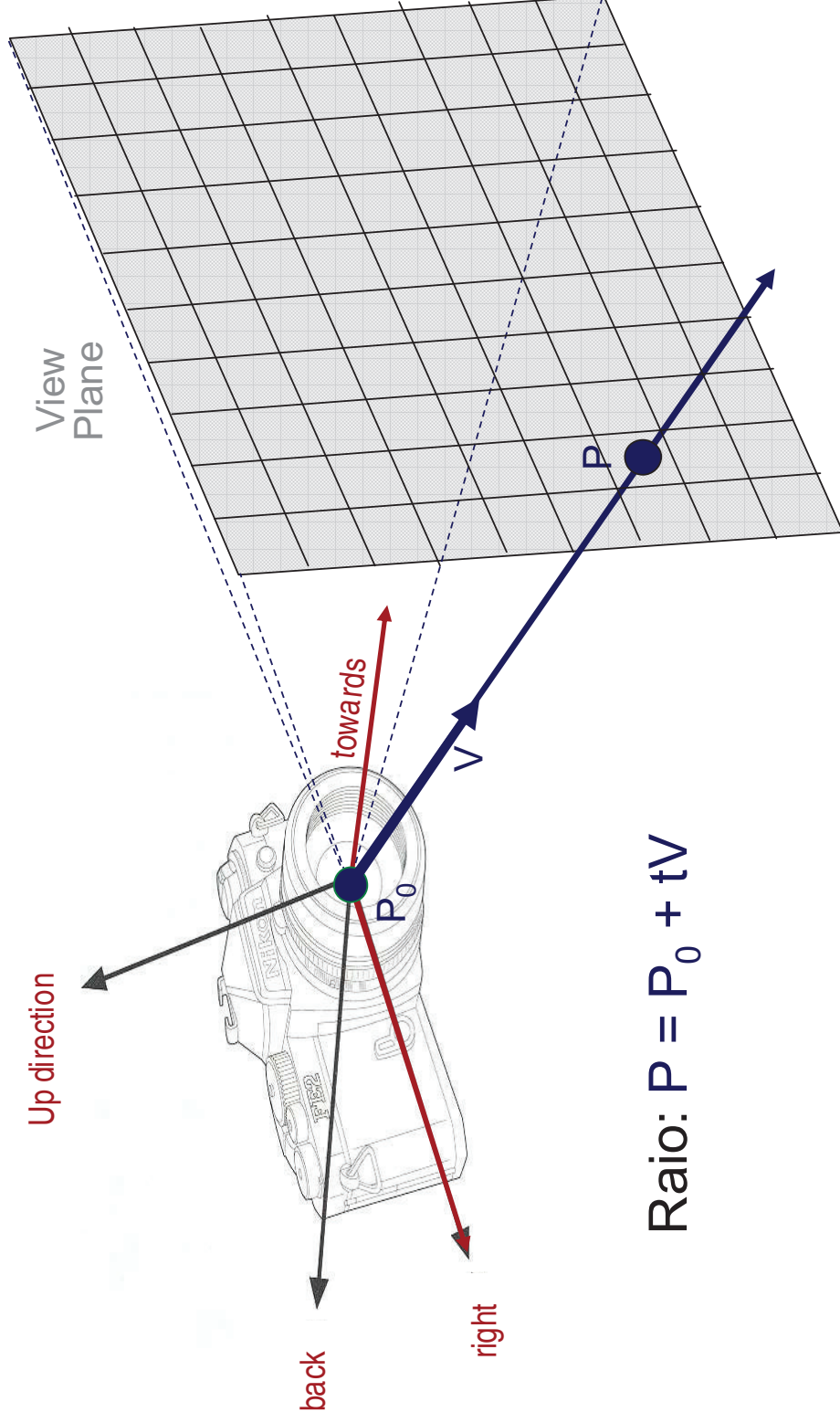


Ray Casting

- Implementação simples:

```
Image Image(Scene scene, int width, int height)
{
    Image image = new Image(width, height);
    for (int i = 0; i < width; i++) {
        for (int j = 0; j < height; j++) {
            Ray ray = ConstructRayThroughPixel(scene.camera, i, j);
            Intersection hit = FindIntersection(ray, scene);
            image[i][j] = GetColor(scene, ray, hit);
        }
    }
    return image;
}
```

Construindo Raio Através de um Pixel

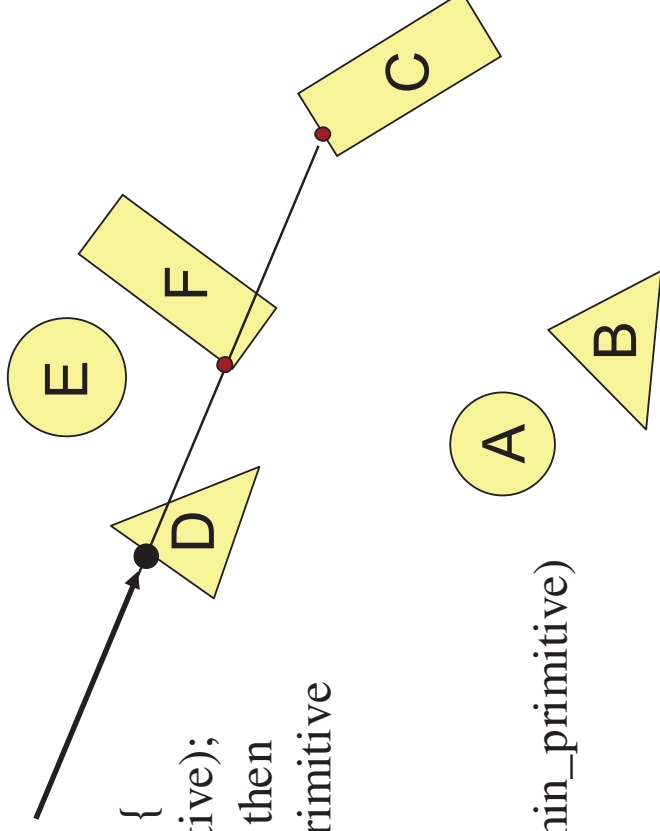


Intersecção Raio-Cena

- Achar intersecção com a primitiva mais próxima

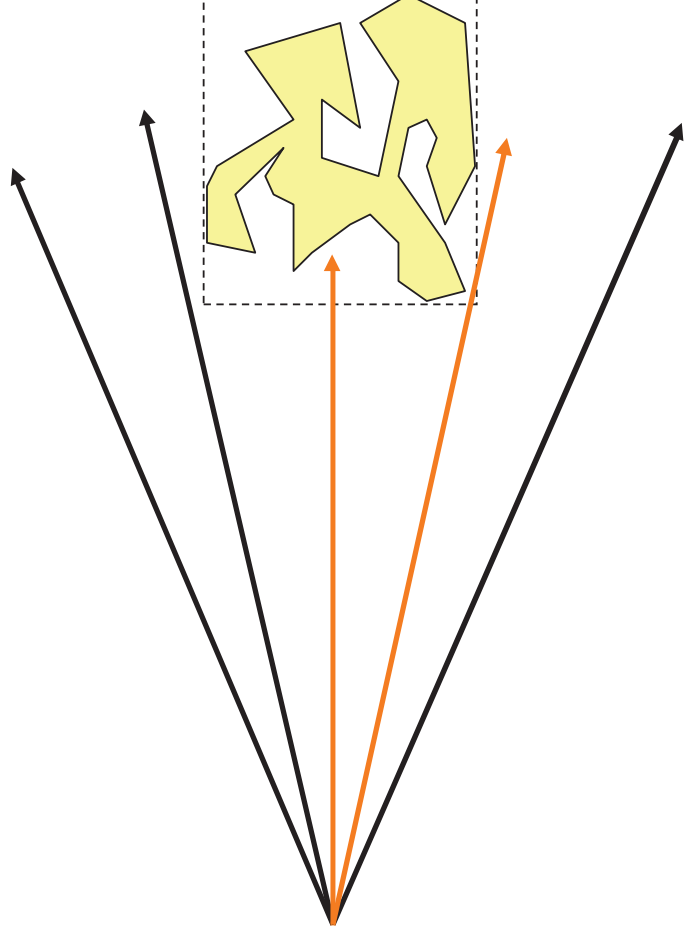
Intersection FindIntersection(Ray ray, Scene scene)

```
{  
    min_t = infinity  
    min_primitive = NULL  
    For each primitive in scene {  
        t = Intersect(ray, primitive);  
        if (t > 0 && t < min_t) then  
            min_primitive = primitive  
            min_t = t  
        }  
    }  
    return Intersection(min_t, min_primitive)  
}
```



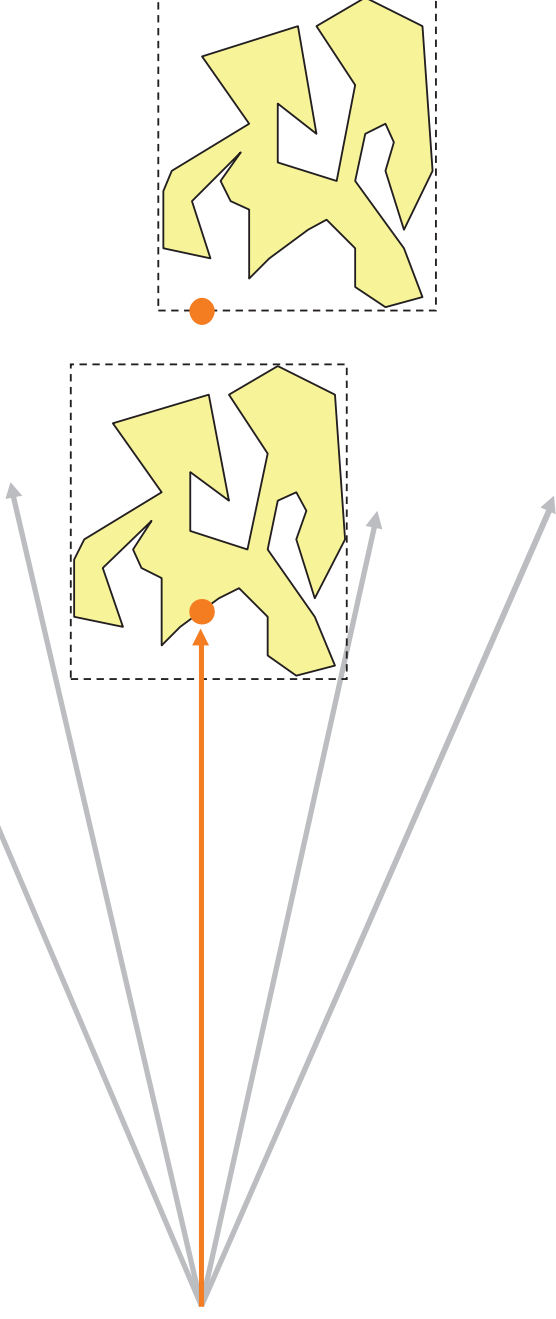
Volumes Limítrofes

- Verificar primeiro intersecção com forma mais simples
 - Se o raio não intersecta o volume limítrofe, então ele não intersecta o seu conteúdo



Volumes Limítrofes

- Verificar primeiro intersecção com forma mais simples
 - Se o raio não intersecta o volume limítrofe, então ele não intersecta o seu conteúdo
 - Se encontrar intersecção mais próxima que a com o limite, então pode-se ignorar todo o conteúdo desse limite



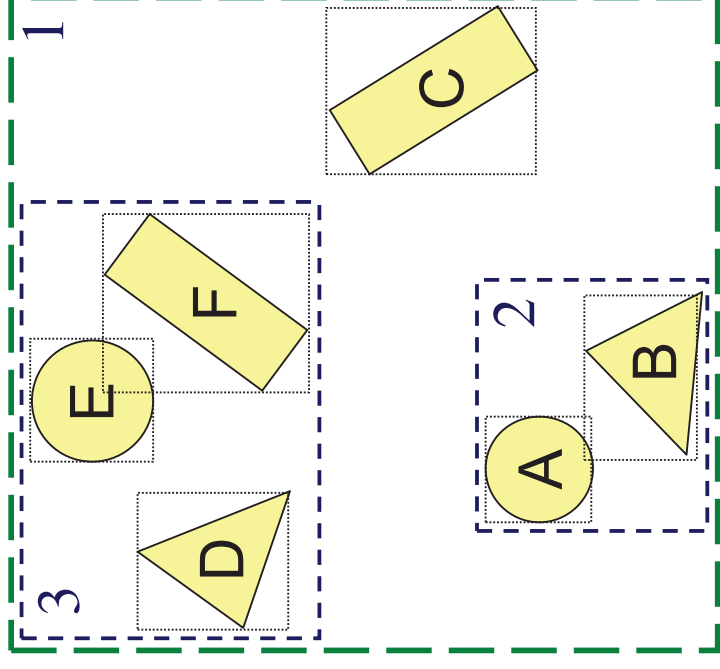
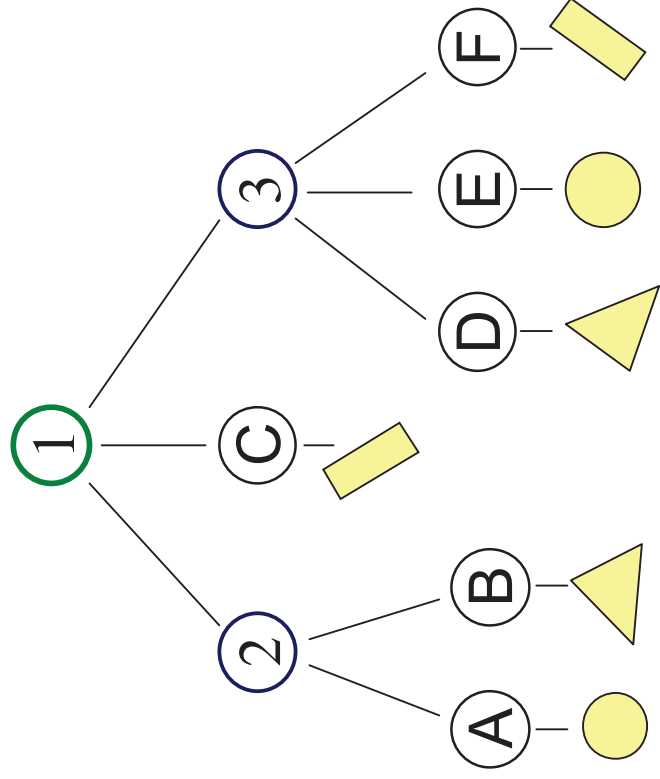
Volumes Limitrofes

- Ordenar "hits" & detectar terminação prematura

```
FindIntersection(Ray ray, Scene scene)
{
    // Find intersections with bounding volumes
    ...
    // Sort intersections front to back
    ...
    // Process intersections (checking for early termination)
    min_t = infinity;
    for each intersected bounding volume i {
        if (min_t < bv_t[i]) break;
        shape_t = FindIntersection(ray, bounding volume contents);
        if (shape_t < min_t) { min_t = shape_t;}
    }
    return min_t;
}
```

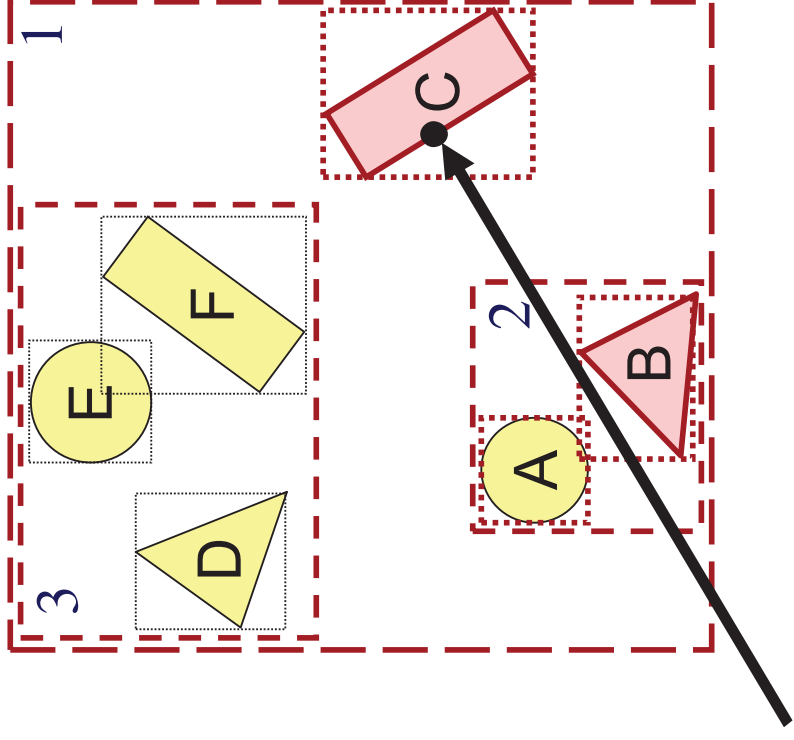
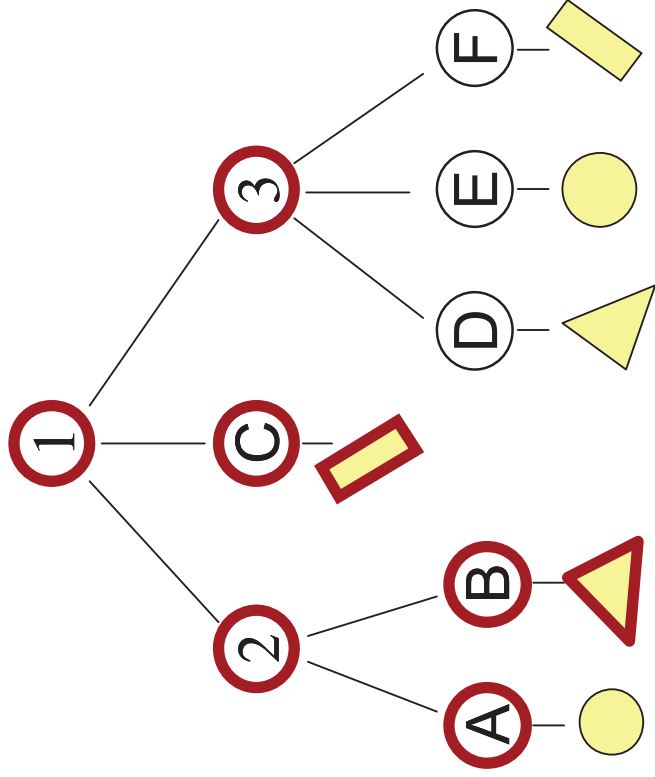
Hierarquias de Volumes Limitrofes I

- Construir hierarquia de volumes limítrofes
 - Volume limítrofe de nó interior contém todos os filhos



Hierarquias de Volumes Limitrofe II

- Usar hierarquia para acelerar intersecções
 - Testar conteúdo de nó apenas se intersectar volume limitrofe



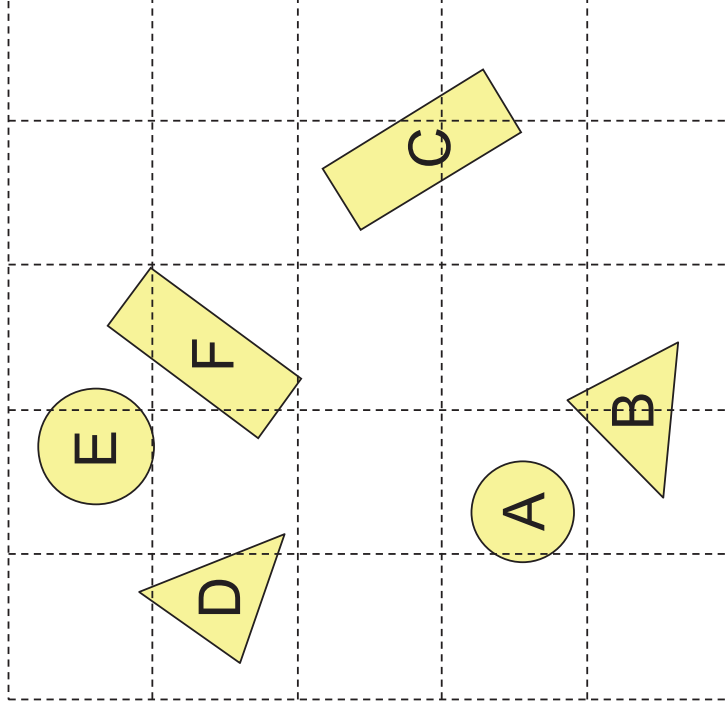
Hierarquias de Volumes Limitrofes III

- Percorrer os nós da cena recursivamente

```
FindIntersection(Ray ray, Node node)
{
    // Find intersections with child node bounding volumes
    ...
    // Sort intersections front to back
    ...
    // Process intersections (checking for early termination)
    min_t = infinity;
    for each intersected child i {
        if (min_t < bv_t[i]) break;
        shape_t = FindIntersection(ray, child);
        if (shape_t < min_t) { min_t = shape_t;}
    }
    return min_t;
}
```

Grid Uniforme

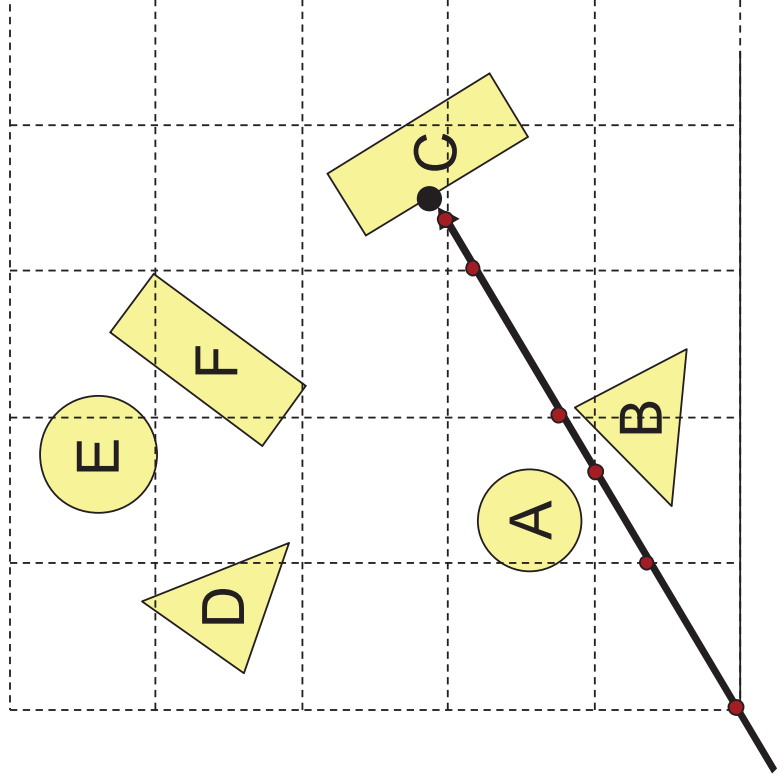
- Construir grid uniforme sobre a cena
 - Indexar primitivas de acordo com a sobreposição das células



Grid Uniforme

- Traçar raios através das células do grid
 - Rápido
 - Incremental

Apenas testa primitivas nas células intersectadas

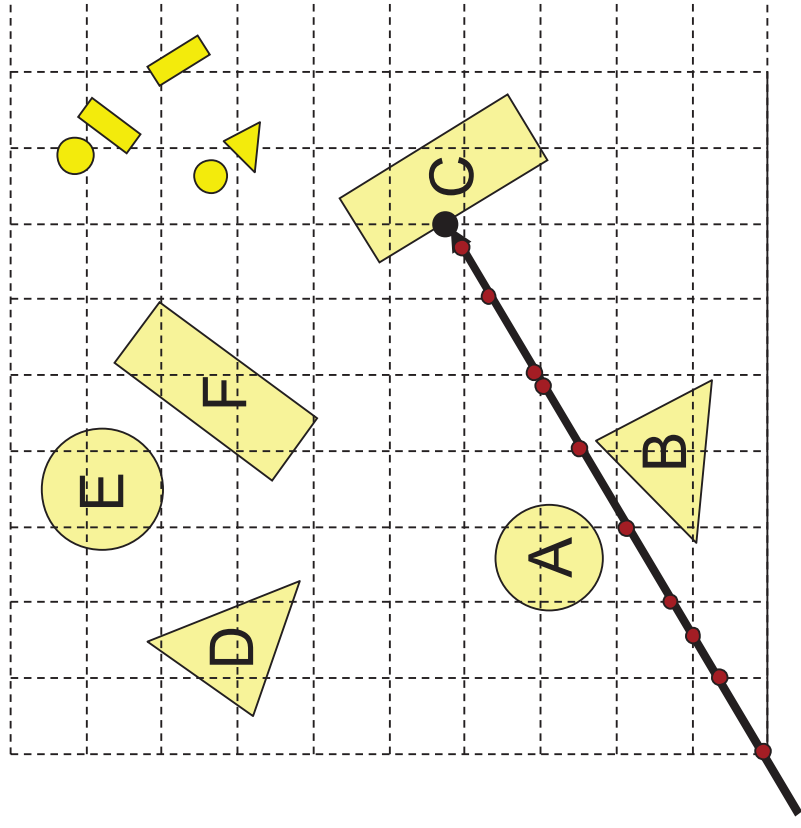


Grid Uniforme

- Problema "potencial":
 - Como escolher uma resolução adequada para o grid ?

Muito pequena
torna o grid esperso

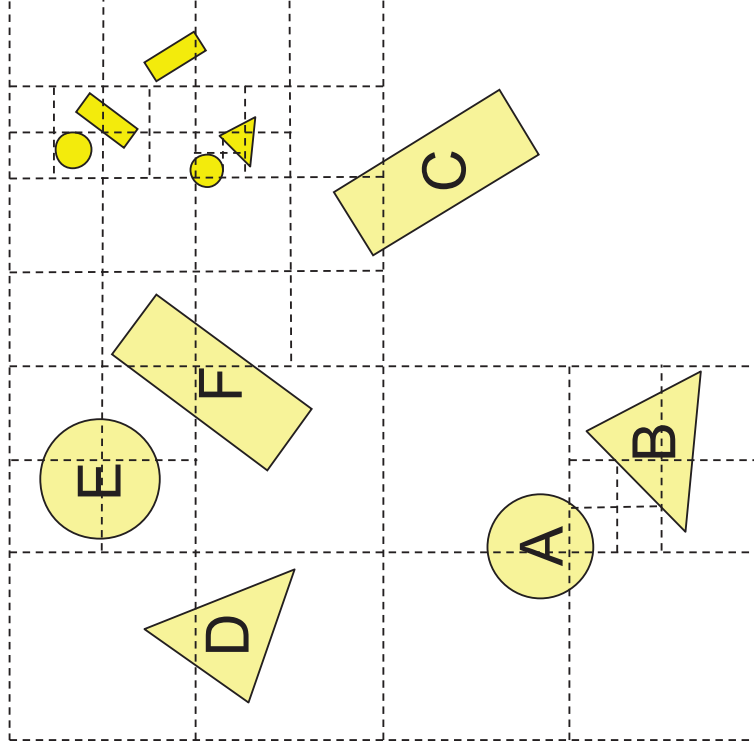
Muito cara se
o grid for denso



Octree

- Construir grid adaptivo sobre a cena
 - Subdividir recursivamente as células em 8 octantes
 - Indexar primitivas a partir da sobreposição com as células

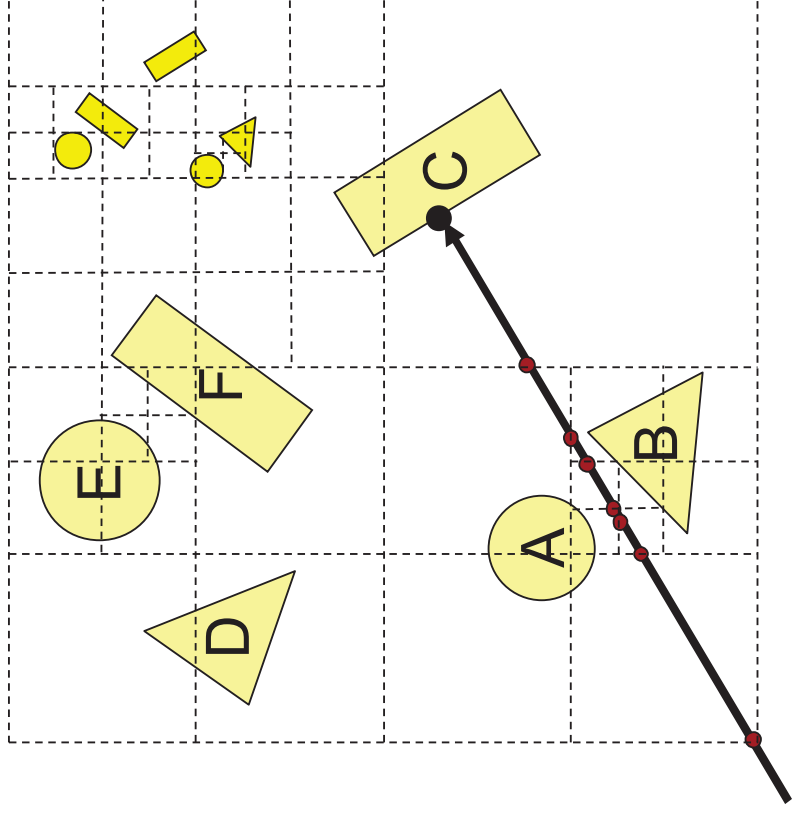
Gera menos células



Octree

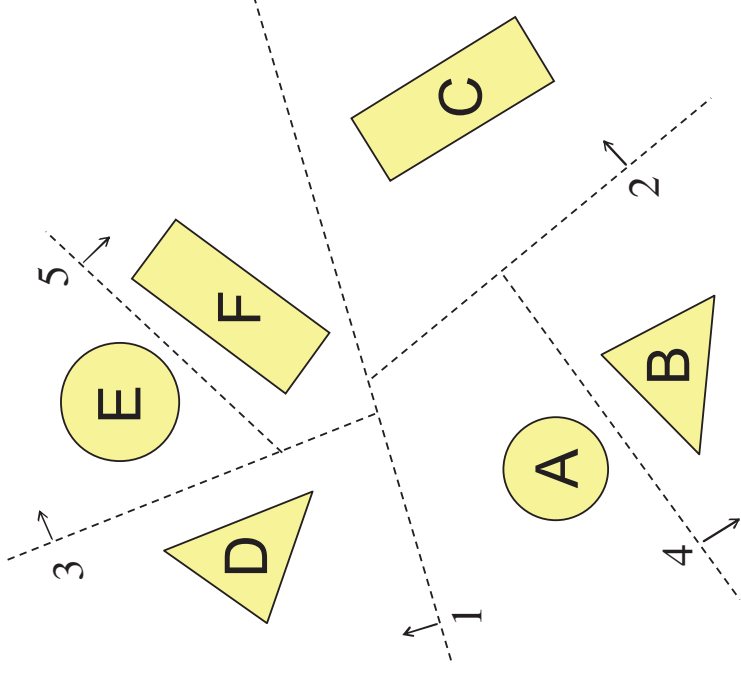
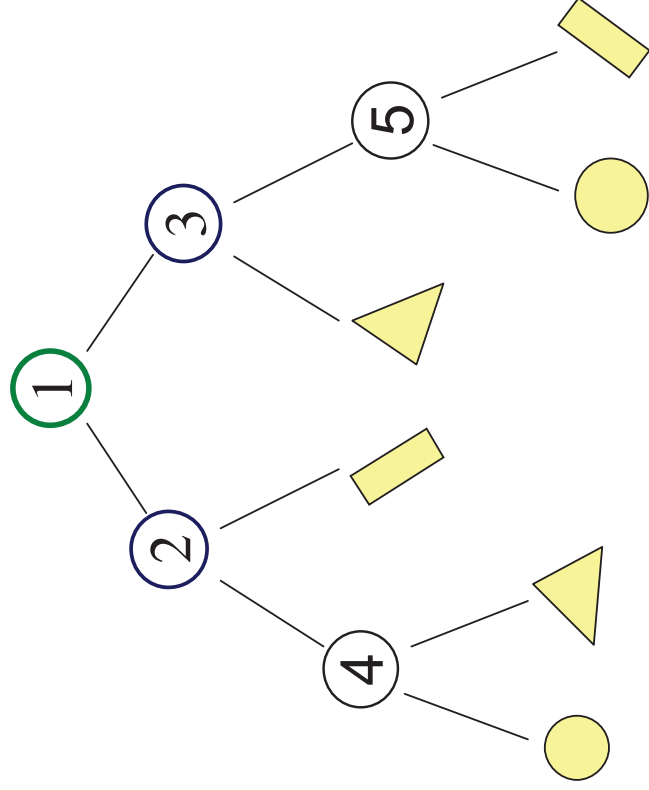
- Traçar raios através das células vizinhas
 - Poucas células
 - Maior complexidade na determinação de vizinhos

Troca menos células por
mais caro caminhamento



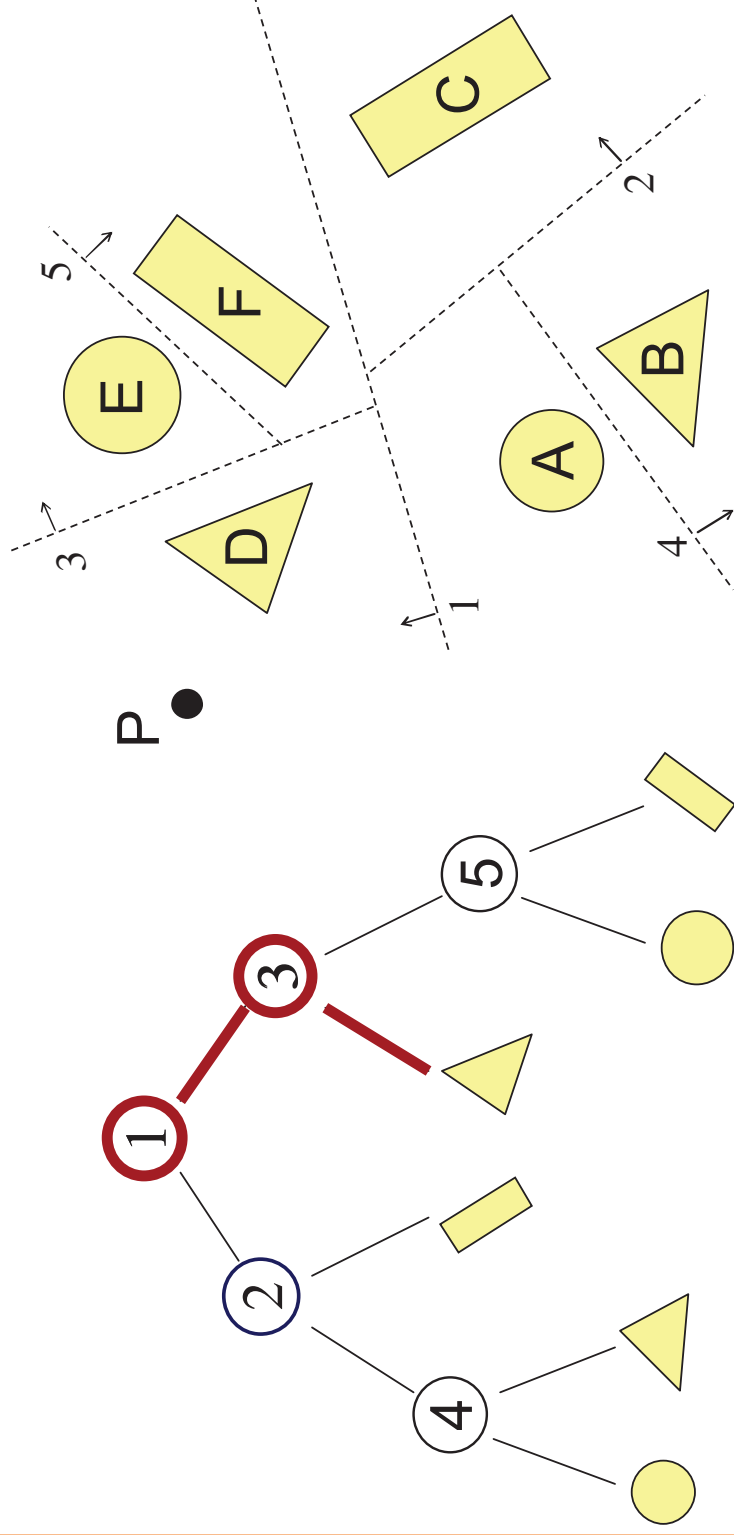
Binary Space Partition (BSP) Tree

- Partição recursiva do espaço por planos
 - Cada célula é um poliedro convexo



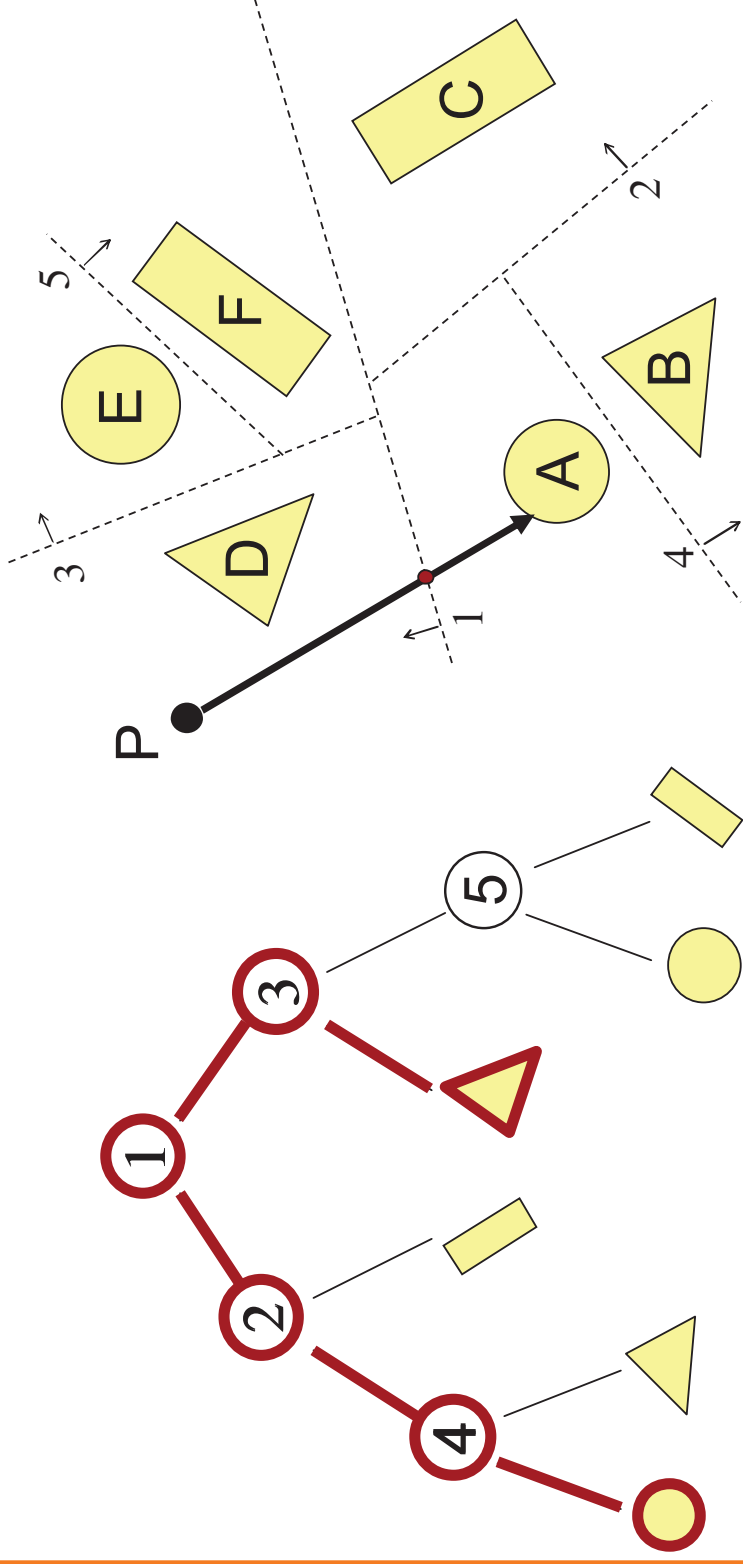
Binary Space Partition (BSP) Tree

- Algoritmos recursivos simples
 - Exemplo: localização de ponto



Binary Space Partition (BSP) Tree

- Traçar raios através da recursão sobre a árvore
 - BSP tree permite caminhamento simples (próximo-distante)



Binary Space Partition (BSP) Tree

```
RayTreeIntersect(Ray ray, Node node, double min, double max)
{
    if (Node is a leaf)
        return intersection of closest primitive in cell, or NULL if none
    else
        dist = distance of the ray point to split plane of node
        near_child = child of node that contains the origin of Ray
        far_child = other child of node
        if the interval to look is on near side
            return RayTreeIntersect(ray, near_child, min, max)
        else if the interval to look is on far side
            return RayTreeIntersect(ray, far_child, min, max)
        else if the interval to look is on both side
            if (RayTreeIntersect(ray, near_child, min, dist)) return ...;
            else return RayTreeIntersect(ray, far_child, dist, max)
}
```

Aceleração

- Técnicas para acelerar interseção são importantes
 - Hierarquias de volumes limitrofes
 - Partições espaciais
- Conceitos gerais
 - Ordenar objetos espacialmente
 - Fazer rejeições triviais o mais rápido possível
 - Utilizar informações de coerência se possível

Tempo esperado é sublinear no núm. de primitivas